Database RPC Mapping

This document explicitly maps the **Remote Procedure Calls (RPCs)** executing the business logic across the 69 tables in the live Supabase database.

(Note: While some tables heavily utilize custom Postgres server-side functions, the majority of the 69 tables are accessed strictly via standard Supabase Javascript Client CRUD operations rather than dedicated stored procedures).

🟤 Core Tables Powered by Active RPCs

These are the primary tables that are actively modified, calculated, or joined by the **6 Deployed SQL Functions** and major documented local functions running inside the database engine.

Table Name	Active RPCs Executing on this Table	Description of Operations
`attendance`	`generate_monthly_payroll()` `clock_in()`	Used to calculate total working days for payroll batch runs, and processed to prevent duplicate daily check-ins.
`employee_finance`	`get_my_profile_v3()` `generate_monthly_payroll()`	Accessed to retrieve fixed base salaries, HRA, and tax deductions during user login and payroll generation.
`leaves`	`process_leave_request()`	Handles the state-flow, approval triggers, and AI risk calculations when an employee requests time off.
`orgs`	`get_my_profile_v3()`	The foundational root joined during the initial login fetch to lock the session to a specific active tenant.
`payroll`	`generate_monthly_payroll()`	The ultimate destination where the looping transaction bulk-inserts newly calculated monthly net payslips.

`profiles`	`calculate_task_points()`
	`get_my_profile_v3()`
	`sync_user_profile()`
	`update_my_profile()`
		Extremely heavy RPC usage. Auth sync triggers copy into it, login fetches it, task gamification updates its stats, and secure self-edits guard it.

`task_submissions`	`calculate_task_points()`	Searched and analyzed to calculate the final gamification score (bonuses/penalties) whenever an employee submits a task.
`task_steps`	`get_task_progress()` `rpc_compute_task_risk_metrics()`	Analyzed mathematically to determine completion ratios and feed progress bars or AI risk monitors.

🟤 Specialized & Trigger-Based RPC Tables

These tables are heavily involved with Triggers or Edge Functions that act similarly to RPCs, or use specific SQL logic mapped in your catalogs.

Table Name	Associated Logic / Functions	Description of Operations
`announcements`	`create_announcement_event()`	Inserts here trigger bulk-loops to generate mapped notifications across the entire organization.
`auth.users` (Supabase)	`sync_user_profile()`	A secure internal Supabase table that fires the sync RPC the moment a new user signs up.
`leave_ai_analysis`	*Deno Edge Function*	Actively populated by external API logic evaluating the text of the leave request.
`task_risk_snapshots`	`rpc_insert_task_risk_snapshot()`	Utilized to store historical AI evaluation arrays when a project's risk parameters are analyzed.
`tasks`	`trg_update_task_hours()` / Edge Functions	Logic fires here whenever an allocated time budget is modified.

🚫 Tables Operating WITHOUT Custom RPCs (Standard CRUD)

The remaining **~56 tables** currently do not rely on custom Postgres stored procedures or RPCs to function. They are operated entirely via standard frontend `supabase.from('table').select/insert()` logic.

Organizational & Projects (Direct CRUD):

- `company_details`
- `departments`
- `project_documents`
- `project_members`
- `projects`
- `team_financials`
- `team_members`
- `teams`

Chat & Messaging (Direct CRUD):

- `attachments`
- `conversation_indexes`
- `conversation_members`
- `conversations`
- `message_reactions`

- `messages`
- `poll_votes`

Hiring & Invoicing (Dormant/Standard CRUD):

- `candidates`
- `clients`
- `interviews`
- `invoice_emails`
- `invoice_items`
- `invoice_templates`
- `invoices`
- `job_descriptions`
- `jobs`
- `offers`

Skills & Audit Logs (Direct CRUD / Inserts):

- `chat_history`
- `document_chunks`
- `documents`
- `employee_monthly_logs`
- `employee_performance_snapshots`
- `employee_reviews`
- `employee_skills`
- `employee_stage_history`
- `extension_requests`
- `feedback`
- `leave_audit_logs`
- `notes`
- `notifications`
- `organization_memberships`
- `payslips`
- `policies`
- `project_time_logs`
- `semantic_cache`
- `skills_master`
- `task_audit`
- `task_evidence`
- `task_feedback`
- `task_notes`
- `task_reviews`
- `task_skills`
- `task_stage_history`
- `task_stages`
- `task_state_history`
- `task_submission_skills`
- `tickets`

- `time_logs`
- `timesheets`

Technical Summary

Rather than putting an RPC on every single table, your architecture wisely restricts heavy Postgres Functions solely to **Atomic Transactions** (like generating payroll loops) and **Secure Calculations** (like task gamification points and auth syncing). All basic data display (chat records, checklists, notes) is handled cleanly by standard client lookups.